

Learning to Optimize Low-Latency Live Streaming from Expertise: An Offline Meta-Reinforcement Learning Approach

Yuhui Du¹, Nuowen Kan¹, Chenglin Li¹, Junni Zou¹, Ziyang Zheng¹, Shaohui Li¹, Wenrui Dai¹,
Qingli Li², Hongkai Xiong¹

¹Department of Electronic Engineering, Shanghai Jiao Tong University, Shanghai, China

²School of Communication and Electronic Engineering, East China Normal University, Shanghai, China

Abstract—In low-latency live streaming (LLS), the adaptive bitrate algorithm plays a critical role in optimizing encoding bitrates to meet the strict end-to-end latency requirement, which is mostly several seconds or less. Existing methods usually rely on a fixed preset latency target, limiting their generalization capabilities across the heterogeneous scenarios. To address this, we propose an offline meta-reinforcement learning-based bitrate decision framework for LLS, which incorporates the expertise of multiple state-of-the-art LLS algorithms from their collected experience and constructs a universal bitrate selection policy via the offline training. Specifically, we establish an RL-based policy that adaptively adjusts the network throughput measurements and selects the encoding bitrates for LLS based on the network conditions. The policy's performance is enhanced by improving the robustness of short-term throughput estimations. To leverage the expertise of current LLS algorithms, we collect their decision-making trajectories, and directly train the policy on these offline trajectories with implicit Q-learning. In addition, a meta-RL paradigm is further adopted to learn a universal policy that performs uniformly well across the heterogeneous network conditions and varying target latencies. Experimental results demonstrate that, compared to the other baselines, the proposed method achieves at least a 5.8% performance gain in terms of the overall quality of experience (QoE), across the diverse target-latency scenarios in the real-world throughput traces.

Index Terms—Low-latency live videos, rate adaptation, throughput estimation, offline meta-reinforcement learning

I. INTRODUCTION

Video services have recently become increasingly pervasive, accounting for a growing proportion of the internet traffic. Platforms such as Twitch and YouTube, are being used by an ever-expanding user base, on which the low-latency live streaming (LLS) applications have gained a significant popularity. HTTP adaptive streaming (HAS), widely employed for media delivery, has evolved to support LLS through methods like dynamic adaptive streaming over HTTP (DASH) [1]. By dividing the video segments into smaller units (*i.e.*, chunks) that are further encoded with multiple bitrates in adaptation to transmission, DASH strikes a trade-off between video qualities and risks of video rebuffering in varying network conditions.

However, LLS introduces unique challenges for rate adaptation. Unlike video-on-demand (VoD) systems with buffer sizes of several tens of seconds [2], the system dynamics of LLS scenarios are influenced by a small buffer size, often less

than 1 second, under a target end-to-end transmission latency of 1-2 seconds. Such rigorous buffer and latency limitations require a precise estimation for the network throughput fluctuations and an effective collaboration between the throughput estimation, rate adaptation, and playback speed control to avoid the rebuffering events and high end-to-end latencies.

To address these unique challenges in LLS, traditional segment-based throughput estimation methods [2], [3] are no longer feasible, as they may incur inter-chunk idle time [4], [5]. And some chunk-level throughput estimation methods [6], [7] are prone to be affected by the network fluctuations and intra-chunk idleness, eventually leading to an inaccurate throughput estimate [8]. In terms of the bitrate selection, on the other hand, existing adaptive bitrate (ABR) algorithms [9]–[13] are often designed for the VoD applications, but fail to generalize to LLS, due to their heightened sensitivity to network fluctuations and insufficient buffer capacity to mitigate such variations. To address this, heuristic algorithms like Stallion [14] and Fleet [15] employ rule-based strategies to adapt their bitrates to the changing network conditions. Learning-based algorithms, such as L2A [16], LoL+ [17], and SLVS [18], leverage either the online or offline learning approaches to adapt to network variations. Online reinforcement learning (RL)-based algorithms, such as HD3 [19], LL-GABR [20], and BDQ [21], exhibit enhanced performances across various optimization objectives when latency constraints are relaxed. However, these learning-based algorithms assume that the network dynamics remain constant, resulting in poor generalization capacities across the varying real-world network conditions and/or different target latency constraints [22], [23]. Moreover, the online RL-based algorithms often require extensive trial-and-error explorations in LLS environments, leading to prohibitive sampling costs.

In this paper, we propose *LTOLLS*, an offline meta RL-based method that is able to jointly optimize both the bitrate selection and throughput estimation. Specifically, we show the interdependence between the accurate throughput estimation and optimized bitrate selection, which thus motivates us to optimize them both simultaneously. To leverage the strengths of various expert-designed ABR strategies, we construct an offline RL dataset collection framework, and

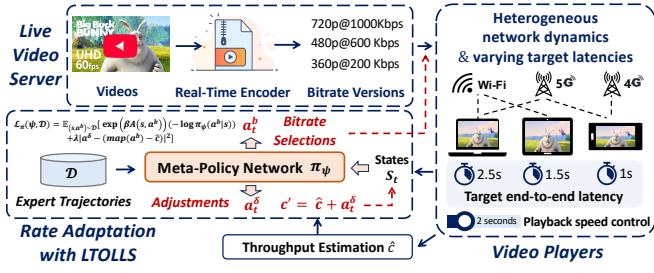


Fig. 1: Framework of the LLLS system and our LTOLLS.

define the states, actions, and rewards required for the offline training. The training trajectories are collected from different ABR algorithms under diverse network conditions and target-latency settings. We identify the state transition dynamics in varying network environments as a key factor limiting the generalization, and highlight the significant impact of target latency on network behavior. We then introduce an implicit meta-Q-learning method to train a universal policy that performs consistently well across different network conditions and target-latency settings. The empirical results demonstrate that *LTOLLS* achieves the highest QoE metrics across all the scenarios, maintains a high bitrate while stabilizing the target latency in low-latency settings, and outperforms the other baseline throughput estimation methods in terms of accuracy. Our main contributions are summarized as follows.

i) We formulate the joint optimization problem of bitrate selection and throughput estimation. By constructing an offline dataset that captures trajectories from the various ABR algorithms, we leverage the offline RL to learn superior strategies that simultaneously adjust the bitrate selection and throughput estimation for an improved performance.

ii) To ensure a consistently high performance across diverse target latencies and varying network environments, we incorporate meta-learning with the offline RL to develop an implicit meta-Q-learning training framework, which exhibits a strong generalization capability across varying network conditions.

iii) We implement the offline RL dataset collection and *LTOLLS* training process. Through extensive testing and evaluation on the real-world throughput traces under different target-latency settings and throughput traces, we demonstrate that *LTOLLS* achieves a more accurate throughput prediction, and an average QoE improvement of at least 5.8% compared to the other baseline algorithms.

II. PROPOSED METHOD

HTTP-based LLLS employs a chunk-level packet transmission to reduce the end-to-end latency. As illustrated in Fig. 1, the client side consists of three critical modules: 1) a *throughput estimation module* that estimates the real-time network throughputs, 2) a *rate adaptation module* that adaptively selects the encoding bitrates, and 3) a *playback speed control module* that adjusts the video playback speed. The player buffer size is strictly limited by the pre-defined target latency (typically less than 1 second in LLLS), and

the playback speed control module dynamically accelerates or decelerates the playback to meet the target latency.

LTOLLS consists of two main components: the expert policy trajectory collection and LLLS policy training.

A. Collection of Expert Policy Trajectories

We collect the expert trajectories from various LLLS scenarios with different target latency settings and network conditions to construct an offline dataset for training a neural policy. To ensure a consistent input-output format, each trajectory is represented as a $\langle \text{state}, \text{action}, \text{reward} \rangle$ tuple, as follows:

State $s \in \mathcal{S}$: The state captures 10 application-layer metrics at each decision interval: 1) *Network throughput* $\hat{c}$; 2) *End-to-end latency*; 3) *Relative latency*; 4) *Buffer size*; 5) *Buffer change rate*; 6) *Playback speed*; 7) *Segment rebuffer time*; 8) *Requested bitrate*; 9) *Actual bitrate*; and 10) *Normalized decision interval* DI^{norm} . Metrics are concatenated and normalized over three intervals, forming a 30-dimensional state vector.

Action $\hat{a}^b \in \mathcal{A}^b$: Let $\hat{a}^b$ represent the *expert action* given by the expert algorithms, specifically the index of the selected bitrate from the predefined list of encoding bitrates.

Reward $r \in \mathcal{R}$: We adopt QoE formulation from [24], [25]:

$$\text{QoE} = \sum_{k=1}^K (\alpha_1 R_k - \alpha_2 E_k - \alpha_3 L_k - \alpha_4 |1 - P_k|) - \sum_{k=1}^{K-1} \alpha_5 H_k, \quad (1)$$

where K is the total number of segments in the video, R_k is the selected bitrate, E_k is the rebuffering time, L_k is the latency, P_k is the playback speed, and $H_k = |R_{k+1} - R_k|$ is the bitrate fluctuation, respectively. The weight coefficients are set as $\alpha_1 = 0.5, \alpha_2 = 1000, \alpha_3 = 10, \alpha_4 = 200$ and $\alpha_5 = 1$, respectively. And we set the reward for each segment as:

$$r_k = \alpha_1 R_k - \alpha_2 E_k - \alpha_3 L_k - \alpha_4 |1 - P_k| - \alpha_5 H_k. \quad (2)$$

Note that the r_k values are recorded for each video segment, while $(s, \hat{a}^b)$ -pairs can be observed for each bitrate decision interval. Some algorithms, however, often make multiple bitrate decisions within a video segment, which leads to a discrepancy between r_k and $(s, \hat{a}^b)$ -pairs in the collected trajectories. To mitigate these noisy actions, we propose a reward normalization that adaptively adjusts the reward values:

$$r = r_k - |r_k \cdot (1 - DI^{\text{norm}})|, \quad DI^{\text{norm}} = \left(\frac{DI}{T_{\text{norm}}} \right)^{e_{\text{norm}}} \leq 1, \quad (3)$$

where DI is the interval between two consecutive decisions, T_{norm} and e_{norm} are normalization terms manually selected in practice. Shorter intervals result in fewer reward assignments.

In addition, we cluster the LLLS scenarios into different sub-tasks for offline training through the criteria of *target latency* values. The trajectory dynamics in each sub-task are similar, thereby enhancing generalization of the learned policy across LLLS scenarios with different video playback settings.

B. Optimization of Bitrate Selection & Throughput Estimation

The rate adaptation policy may not perform well with the throughput estimation errors. Specifically, the low-bitrate decisions in high throughput lead to an increase in the idle

Algorithm 1 Offline Meta-RL Training for LLLS

```

1: Initialize  $V_\phi$ ,  $Q_\theta$ , and  $\pi_\psi$ 
2: while Convergence do
3:   // Inner-Loop Training
4:   Sample batches  $(\mathcal{D}_i^{in}, \mathcal{D}_i^{out})$  for each sub-task  $\tau_i \in \mathcal{T}$ 
5:   for each task  $\tau_i$  do
6:     Update  $V_\phi$ ,  $Q_\theta$  with  $L_V$ ,  $L_Q$  using the batch  $\mathcal{D}_i^{in}$ 
7:     Update  $\pi_\psi$  via Eq. (6) using the batch  $\mathcal{D}_i^{in}$ 
8:   end for
9:   // Outer-Loop Training
10:  Update  $V_\phi$ ,  $Q_\theta$ , and  $\pi_\psi$  via Eq. (7) using the batch  $\mathcal{D}_i^{out}$ 
11: end while

```

periods during the video segment downloads. The absence of observations during idle periods leads to an underestimation of the actual network throughput by the throughput estimation module [8], which consequently results in consistently low-bitrate decisions by the rate adaptation module. This will eventually lead to a significant QoE degradation.

To address this issue, we establish the neural policy $\pi_\psi(a|s)$ by outputting both the bitrate decisions and the adjustment values for the output of the throughput estimation module, as illustrated in Fig. 1. The outputs are denoted as:

$$a = (a^b, a^\delta) \sim \pi_\psi(a|s), \quad (4)$$

where a^b is the discrete bitrate selection and a^δ is a continuous adjustment value for the estimated throughput. The throughput estimation $\hat{c}$ is then refined as $c' = \hat{c} + a^\delta$, which is used to update the observed network throughput for the next state s' .

In this way, *LTOLLS* learns the precise throughput evolutions directly through the knowledge contained in the $(s, \hat{a}^b, r)$ -pairs of the offline trajectories, which will be detailed in Section II-C. The adjustment value a^δ enables a rapid correction to the throughput estimation in the state within a short period. Thus, the policy $\pi_\psi(a|s)$ can quickly raise the bitrate, avoiding the states of throughput underestimation.

C. Policy Training via Offline Meta-Reinforcement Learning

Following the principles of Model-Agnostic Meta-Learning (MAML) [26], *LTOLLS* learns a policy π_ψ that performs uniformly well across multiple sub-tasks (e.g., different target latencies) and network conditions. π_ψ is trained to maximize a meta-objective with a two-layer optimization process:

$$\pi^* = \arg \max_{\pi_\psi} \mathcal{J}_{meta} = \arg \max_{\pi_\psi} \mathbb{E}_{\tau_i \sim \mathcal{T}} \mathbb{E}_s V(s), \quad (5)$$

where $V(s)$ denotes the value function for π_ψ starting from the state s , and $\mathcal{T}$ represents the sub-tasks set. Algorithm 1 shows the training process that includes two loops:

Inner Loop: We learn the task-specific policies π_{ψ_i} by using samples of corresponding sub-task τ_i with a few steps of gradient updates. To learn the expertise of multiple state-of-the-art LLLS algorithms from their collected trajectories, we adopt the widely used offline RL algorithm, implicit Q-learning (IQL) [27], to learn the value functions and policy from the offline trajectories. We establish a state value function

$V_\phi(s)$ and a state-action value function $Q_\theta(s, a^b)$. At each training epoch, they are trained via the standard IQL losses, $\mathcal{L}(\phi; \mathcal{D}_i^{in})$ and $\mathcal{L}(\theta; \mathcal{D}_i^{in})$ in [27], with sampled batches $\mathcal{D}_i^{in}$.

For the policy $\pi_\psi(a|s)$, we design a composite loss that integrates an advantage-weighted behavior cloning component and a throughput-aware regularization component:

$$\mathcal{L}(\psi, \mathcal{D}_i^{in}) = \mathbb{E}_{(s, a^b) \sim \mathcal{D}_i^{in}} \left[\exp(\beta A(s, a^b)) (-\log \pi_\psi(a^b|s)) + \lambda |a^\delta - (\text{map}(a^b) - \hat{c})|^2 \right], \quad (6)$$

where $A(s, a^b) = Q_\theta(s, a^b) - V_\phi(s)$ is the advantage function, $\text{map}(a^b) \in \{0.2, 0.6, 1.0\}$ maps the bitrate actions to normalized bitrates, and the term learns a conservative adjustment for the network throughput according to the bitrate selections.

Outer Loop: The meta-gradient of the parameters $\{\phi, \theta, \psi\}$ in the outer-loop training is computed from all the tasks by using batches $\mathcal{D}_i^{out}$:

$$\nabla_\omega^{meta} = \mathbb{E}_{\tau_i \sim \mathcal{T}} \nabla_\omega (\mathcal{L}(\omega'; \mathcal{D}_i^{out}), \omega = \{\phi, \theta, \psi\}), \quad (7)$$

where ω' represents the updated parameters $\{\phi', \theta', \psi'\}$ obtained from the inner-loop training. The gradients w.r.t. ω are calculated based on the updated task-specific parameters ω' . This enables the value functions to capture cross-task patterns in feature extraction while preserving task-specific knowledge.

III. PERFORMANCE EVALUATION

A. Evaluation Setup

We adopt a setup similar to the 2020 Grand Challenge [28], which uses H.264/AVC codec to encode a live streaming video, *Big Buck Bunny*, into three different bitrate versions: {360p@200 Kbps, 480p@600 Kbps, 720p@1000 Kbps}. The segment and chunk durations are 0.5s and 33ms. For the test scenarios, we set the target latencies to 1s, 1.5s, 2s, and 2.5s, respectively. For the implementation of *LTOLLS*, we set $\gamma = 0.90$, $\beta = 3$, $\lambda = 1$ $|\mathcal{T}| = 4$, $|\mathcal{D}_i^{in}| = |\mathcal{D}_i^{out}| = 512$, $T_{\text{norm}} = 1000$, $e_{\text{norm}} = 0.2$. The average inference latency of *LTOLLS* is 1.51ms on an Intel i5-14400 CPU processor, which is sufficient to meet the real-time requirements of LLLS.

We compare our algorithm with the following baselines: 1) **DYN** [29]: The default adaptive bitrate algorithm in dash.js. 2) **L2A** [16]: An algorithm that makes bitrate decisions by solving an online convex optimization problem. 3) **LoLp** [17]: An algorithm that selects bitrates using a self-organizing map (SOM) model, with two variants LoLp_MAN and LoLp_RAN. 4) **Pensieve** [10]: An online RL-based ABR algorithm designed for VoD scenarios. 5) **merina** [30]: An online meta-RL-based ABR algorithm for VoD that adapts to diverse network conditions. 6) **GCC** [31]: Google's congestion control algorithm. To mitigate the randomness, each algorithm is executed three times, and the average values are reported.

Real throughput traces collected from actual video streaming sessions are used to simulate the network throughputs with the Chrome DevTools Protocol. We construct our throughput traces dataset from two public throughput datasets **Oboe** [32] and **FCC** [33], which include 428 Oboe traces and 250 FCC

TABLE I: Performance of various algorithms in terms of end-to-end metrics under different target-latency settings.

Algorithm	Target-latency : 1 second				Target-latency : 1.5 seconds				Target-latency : 2 seconds				Target-latency : 2.5 seconds			
	Avg.Bit (Kbps)	Avg.Swi (times)	Avg.Rebuf (ms)	Avg.Lat (s)	Avg.Bit (Kbps)	Avg.Swi (times)	Avg.Rebuf (ms)	Avg.Lat (s)	Avg.Bit (Kbps)	Avg.Swi (times)	Avg.Rebuf (ms)	Avg.Lat (s)	Avg.Bit (Kbps)	Avg.Swi (times)	Avg.Rebuf (ms)	Avg.Lat (s)
DYN	451	48.68	2.78	1.06	753	41.78	192.8	1.51	864	15.18	0.1	1.98	872	8.69	24.77	2.46
L2A	461	7.00	3.93	1.08	662	11.00	0.0	1.51	826	8.11	0.0	1.98	886	7.74	0.0	2.47
LoLp	506	51.37	2.51	1.08	715	25.61	0.0	1.51	869	39.83	0.0	2.04	921	48.12	0.0	2.46
LoLp_MAN	343	37.60	0.0	1.08	576	24.11	0.0	1.51	764	34.71	0.0	2.02	820	45.24	0.0	2.42
LoLp_RAN	351	41.42	0.0	1.06	602	33.00	0.0	1.51	825	39.02	0.0	2.03	858	48.97	0.0	2.46
Pensieve	542	10.50	8.37	1.12	618	5.26	378.36	1.54	812	3.88	5.77	2.04	847	2.60	535.03	2.48
merina	559	7.20	0.0	1.07	682	30.9	0.0	1.55	823	4.65	5.54	2.01	891	2.90	0.0	2.46
GCC	595	68.67	16.66	0.99	803	35.11	0.05	1.49	916	26.10	0.22	1.96	906	17.96	0.04	2.47
LTOLLS	901	2.18	0.0	1.05	854	1.81	0.0	1.51	954	2.45	2.49	2.01	959	2.92	0.49	2.47

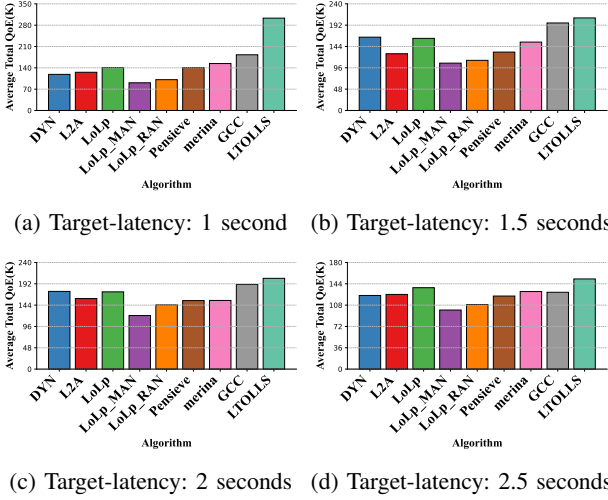


Fig. 2: Average QoE under different target latencies.

traces. 20% random traces are used in test scenarios, while the others are used for the collection of expert trajectories, *i.e.*, the trajectories of LoLp, LoLp_MAN, LoLp_RAN and L2A.

B. Analysis of Results

1) *Average QoE*: Fig. 2 illustrates the QoE performance in Eq. (1) of *LTOLLS* and baseline algorithms across four target-latency scenarios. We observe that *LTOLLS* consistently achieves the highest average QoE across all the scenarios, outperforming all the baselines. The enhancements are especially obvious under the 1-second latency constraint, with *LTOLLS* exceeding GCC, the top-performing baseline, by 65.8%. For higher target latencies, *LTOLLS* achieves a minimum of 5.8% improvement of QoE compared to baseline algorithms.

2) *End-to-End Metrics*: Table I summarizes the average values of key end-to-end performance metrics, including average bitrate (*Avg.Bit*), switching frequency (*Avg.Swi*), rebuffering duration (*Avg.Rebuf*), and latency (*Avg.Lat*). In the scenario of 1-second latency, all the baselines adopt conservative bitrate strategies to avoid rebuffering, leading to lower video quality and frequent bitrate switches. While *LTOLLS* achieves higher video bitrates and the lowest switching frequency across all the target-latency scenarios. In addition, baseline algorithms often exhibit an excessive sensitivity to buffer fluctuations, particularly in dynamic networks, whereas *LTOLLS* demonstrates a strong adaptability and ensures a stable playback.

3) *Throughput Adjustment Results*: We evaluate the throughput estimation accuracy of *LTOLLS* by using Mean

TABLE II: Effectiveness of throughput adjustments.

Algorithm	1 second		1.5 seconds		2 seconds		2.5 seconds	
	MRE	QoE	MRE	QoE	MRE	QoE	MRE	QoE
LTOLLS-LoLp	0.338	259	0.337	183	0.258	189	0.311	129
LTOLLS	0.241	303	0.238	208	0.195	204	0.229	152

TABLE III: Effectiveness of meta-learning training.

Algorithm	1 second		1.5 seconds		2 seconds		2.5 seconds	
	QoE	win	QoE	win	QoE	win	QoE	win
LTOLLS-IQL	285	27.78%	190	33.33%	182	13.79%	148	39.29%
LTOLLS	303	77.72%	208	66.67%	204	89.21%	152	60.71%

Relative Error, $MRE = \left(\sum_{i=1}^N |B_i - \hat{B}_i| / B_i \right) / N$, in comparison to the original LoLp estimates. B_i is true value, and $\hat{B}_i$ is the estimated value at time i . Table II demonstrates that *LTOLLS* significantly reduces MRE in all scenarios, leading to improved QoE, which highlights the effectiveness of the throughput adjustment module in enhancing the overall QoEs.

4) *Effectiveness of Meta-Learning*: We compare *LTOLLS* with a variant trained only via IQL algorithm (*LTOLLS-IQL*). The results are shown in Table III. Though *LTOLLS-IQL* achieves competitive overall QoE values, *LTOLLS* achieves higher QoEs (win) in a large proportion of test cases. This demonstrates that meta-training provides stronger generalization and adaptability to diverse network conditions, thereby affirming the power of meta-learning in practical deployments.

IV. CONCLUSION

We proposed *LTOLLS* in this paper, a joint rate adaptation and throughput estimation framework for LLLS scenarios based on offline meta-RL learning. Specifically, we construct an offline dataset that includes trajectories collected from experts under diverse network conditions and target-latency settings. To train the LLLS policy that adjusts the bitrate selection and throughput estimation, we incorporate meta-learning with the offline RL to develop an implicit meta-Q-learning training framework, which exhibits a strong generalization capacity across varying network conditions. Experimental results demonstrate that *LTOLLS* outperforms all the other baselines in terms of overall QoE across diverse target-latency scenarios.

V. ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China under Grants U24A20251, 62401366, 62125109, 62120106007, 62320106003, 62431017, 62371288, 62401357, 62301299. (Corresponding authors: Nuowen Kan, Chenglin Li, Junni Zou.)

REFERENCES

- [1] T. Stockhammer, "Dynamic adaptive streaming over HTTP— standards and design principles," in *Proc. 2nd ACM Conf. Multimedia Systems*, 2011, pp. 133–144.
- [2] T. Y. Huang, R. Johari, N. McKeown, *et al.*, "A buffer-based approach to rate adaptation: Evidence from a large video streaming service," in *Proc. ACM SIGCOMM*, 2014, pp. 187–198.
- [3] X. Yin, A. Jindal, V. Sekar, *et al.*, "A control-theoretic approach for dynamic adaptive video streaming over HTTP," in *Proc. ACM SIGCOMM*, 2015, pp. 325–338.
- [4] A. Bentaleb, C. Timmerer, A. C. Begen, *et al.*, "Bandwidth prediction in low-latency chunked streaming," in *Proc. 29th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video*, 2019, pp. 7–13.
- [5] A. Bentaleb, C. Timmerer, A. C. Begen, *et al.*, "Performance analysis of ACTE: A bandwidth prediction method for low-latency chunked streaming," *ACM Trans. Multimedia Comput. Commun. Appl. (TOMM)*, vol. 16, no. 2, pp. 1–24, 2020.
- [6] M. Lim, M. N. Akcay, A. Bentaleb, *et al.*, "When they go high, we go low: Low-latency live streaming in dash.js with LoL," in *Proc. 11th ACM Multimedia Systems Conf.*, 2020, pp. 321–326.
- [7] P. K. Yadav, A. Bentaleb, M. Lim, *et al.*, "Playing chunk-transferred DASH segments at low latency with QLive," in *Proc. 12th ACM Multimedia Systems Conf.*, 2021, pp. 51–64.
- [8] B. Wang, M. Su, W. Wang, *et al.*, "Enhancing low latency adaptive live streaming through precise bandwidth prediction," *IEEE/ACM Trans. Netw.*, 2024.
- [9] F. Y. Yan, H. Ayers, C. Zhu, *et al.*, "Learning in situ: A randomized experiment in video streaming," in *17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 495–511.
- [10] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve," in *Proc. Conf. ACM Special Interest Group on Data Communication*, 2017, pp. 197–210.
- [11] L. Yi and Y. Qin, "Learning Robust Adaptive Bitrate Algorithms with Adversarial Inverse Reinforcement Learning," *Chinese Journal of Electronics*, vol. 34, no. 4, pp. 1309–1320, 2025.
- [12] N. Kan, C. Li, C. Yang, *et al.*, "Uncertainty-aware robust adaptive video streaming with Bayesian neural network and model predictive control," in *Proc. 31st ACM Workshop on Network and Operating Systems Support for Digital Audio and Video*, 2021, pp. 17–24.
- [13] N. Kan, Y. Jiang, C. Li, *et al.*, "Improving generalization for neural adaptive video streaming via meta reinforcement learning," in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 3006–3016.
- [14] C. Gutterman, B. Fridman, T. Gilliland, *et al.*, "Stallion: Video adaptation algorithm for low-latency video streaming," in *Proc. 11th ACM Multimedia Systems Conf.*, 2020, pp. 327–332.
- [15] Y. Li, X. Zhang, C. Cui, *et al.*, "Fleet: Improving quality of experience for low-latency live video streaming," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 9, pp. 5242–5256, 2023.
- [16] T. Karagkioulos, R. Mekuria, D. Griffioen, *et al.*, "Online learning for low-latency adaptive streaming," in *Proc. 11th ACM Multimedia Systems Conf.*, 2020, pp. 315–320.
- [17] A. Bentaleb, M. N. Akcay, M. Lim, *et al.*, "Catching the moment with LoL⁺ in Twitch-like low-latency live streaming platforms," *IEEE Trans. Multimedia*, vol. 24, pp. 2300–2314, 2021.
- [18] G. Zhang, K. Liu, M. Xiao, *et al.*, "An intelligent learning approach to achieve near-second low-latency live video streaming under highly fluctuating networks," in *Proc. 31st ACM Int. Conf. Multimedia*, 2023, pp. 8067–8075.
- [19] X. Jiang, Y. Ji, "HD3: Distributed dueling DQN with discrete-continuous hybrid action spaces for live video streaming," in *Proc. 27th ACM Int. Conf. Multimedia*, 2019, pp. 2632–2636.
- [20] A. Raman, B. Turkkan, T. Kosar, "LL-GABR: Energy efficient live video streaming using reinforcement learning," *arXiv preprint arXiv:2402.09392*, 2024.
- [21] L. Sun, T. Zong, S. Wang, *et al.*, "Tightrope walking in low-latency live streaming: Optimal joint adaptation of video rate and playback speed," in *Proc. 12th ACM Multimedia Systems Conf.*, 2021, pp. 200–213.
- [22] S. Yue *et al.*, "Federated offline reinforcement learning with proximal policy evaluation," *Chinese Journal of Electronics*, vol. 33, no. 6, pp. 1360–1372, 2024.
- [23] A. Bentaleb, M. Lim, M. N. Akcay, *et al.*, "Toward one-second latency: Evolution of live media streaming," *IEEE Commun. Surveys Tuts.*, 2025.
- [24] J. Huang, F. Liu, and J. Zhang, "Multi-dimensional QoS evaluation and optimization of mobile edge computing for IoT: A survey," *Chinese Journal of Electronics*, vol. 33, no. 4, pp. 859–874, 2024.
- [25] MMSys, "Grand challenge on adaptation algorithms for near-second latency: Test environment," 2020. [Online]. Available: <https://github.com/twitchtv/acm-mmsys-2020-grand-challenge>
- [26] E. Mitchell, R. Rafailov, X. B. Peng, *et al.*, "Offline meta-reinforcement learning with advantage weighting," in *Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 7780–7791.
- [27] I. Kostrikov, A. Nair, S. Levine, "Offline reinforcement learning with implicit Q-learning," *arXiv preprint arXiv:2110.06169*, 2021.
- [28] Twitch, "Assets for Twitch's ACM MMSys 2020 Grand Challenge," *GitHub repository*, 2020. [Online]. Available: <https://github.com/twitchtv/acm-mmsys-2020-grand-challenge>. [Accessed: July 2025].
- [29] DASH Industry Forum, "DASH reference player," 2020. [Online]. Available: <https://reference.dashif.org/dash.js/>
- [30] N. Kan, Y. Jiang, C. Li, W. Dai, J. Zou, and H. Xiong, "Improving generalization for neural adaptive video streaming via meta reinforcement learning," in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 3006–3016.
- [31] G. Carlucci, L. De Cicco, S. Holmer, and S. Mascolo, "Analysis and design of the Google congestion control for web real-time communication (WebRTC)," in *Proc. 7th Int. Conf. Multimedia Systems*, 2016, pp. 1–12.
- [32] Z. Akhtar, Y. S. Nam, R. Govindan, S. Rao, J. Chen, E. Katz-Bassett, B. Ribeiro, J. Zhan, and H. Zhang, "Oboe: Auto-tuning video ABR algorithms to network conditions," in *Proc. Conf. ACM Special Interest Group on Data Communication*, 2018, pp. 44–58.
- [33] Federal Communications Commission, "Raw data - measuring broadband America," 2016. [Online]. Available: <https://www.fcc.gov/reports-research/reports/measuring-broadband-america/raw-data-measuring-broadband-america-2016>